

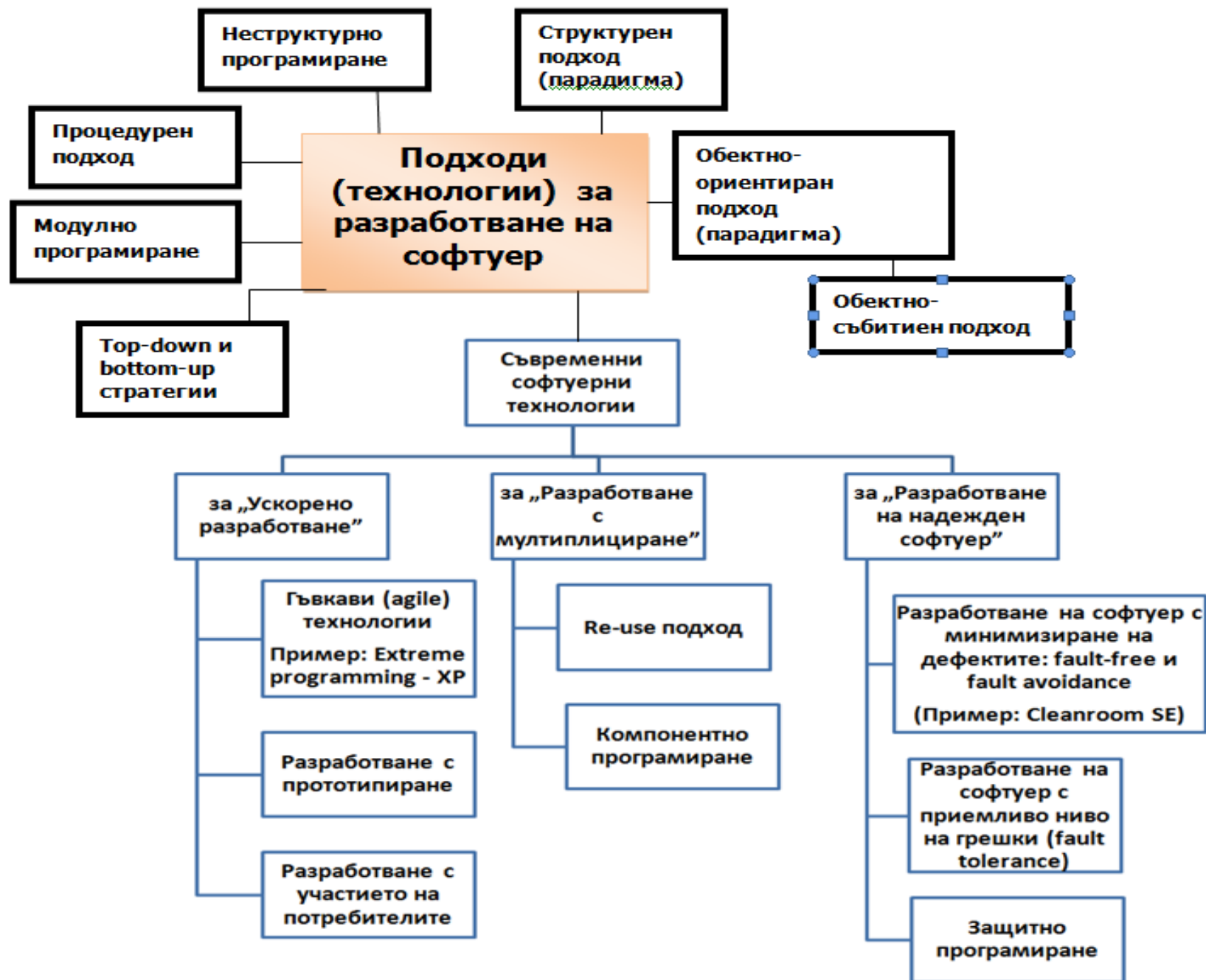
Тема 2. Подходи (технологии) за разработване на софтуер.

дисциплина: Технология на софтуерното производство

лектор: доц. В.Божикова

Съдържание

1. Подходи за разработване на софтуер – препоставки за създаване
2. Изисквания към съвременните подходи (технологии) за разработване на софтуер
3. Неструктурно програмиране (Unstructured Programming)
4. Процедурен подход за разработка на софтуер
5. Модулно програмиране (Modular programming)
6. Top-down и bottom-up design
7. Структурен подход за разработване на софтуер
8. Обектно-ориентиран подход за разработване на софтуер
 - Обектно-събитийната парадигма (ОСП)



1. Подходи за разработване на софтуер – препоставки за създаване

- Проучването на специализираната литература показва, че **продължава създаването на нови модели на ЖЦ**. Поради стремежа да обхванат най-различни аспекти (технологични, организационни и икономически) новосъздаваните модели са **все по-сложни и оттам** — все по-малко приложими в практиката [1],[2].
- Затова, в последно време усилията са насочени по-скоро към създаване и използване на нови **подходи (технологии) за разработване на софтуер**.
- Д. Един подход за разработване на софтуер е **съвкупност от практики, правила, процедури, както и други подходи**, които трябва да се следват през ЖЦ на ПП, с цел – **ефективно** разработване при зададените условия на ПП, с необходимими свойства.

Парадигми

- Д. Най-общите, фундаментални подходи за разработване на софтуер се наричат **парадигми**. Утвърдили са се два такива подхода (парадигми):
 - **структурен подход** за разработване на софтуер (счита се за традиционен или конвенционален) и
 - **обектно ориентиран подход** за разработване на софтуер.

Парадигми

- Казахме, че подходите за разработване на софтуер могат да съдържат от своя страна други подходи, подкрепящи отделните фази на жизнения цикъл на софтуера.
- Пример: **структурния подход за разработка на софтуер**, който ще разгледаме по-подробно се базира на идеите за:
 - „**Модулност**” на програмите и проектите (през фазата на проектиране) , т.е на **модулния подход (МП)**;
 - **Функционална декомпозиция и постъпкова детайлизация** на проблема, чрез прилагане на **подход за проектиране “Отгоре-Надолу”** (Top-Down Design);
 - „**Структурно програмиране**” (през фазата на разработка) и др.

2. Изисквания към съвременните подходи (технологии) за разработване на софтуер

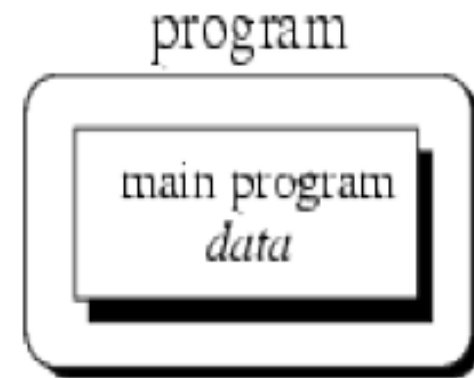
Анализът на известните и широко разпространените подходи (технологии) за проектиране позволява да се формулират следните основни изисквания:

- **свеждане на човешкия фактор до минимум.**
- **насоченост към колектив от програмисти, а не към отделни личности.**
- **освобождение от канцеларщината.**
- **стремеж за автоматизация на всички етапи от работата на колектива от програмисти.**
- **независимост от езика за програмиране**
- **осигуряване на средства за автоматическо фиксиране в хронологичен ред на всички действия използвани в процеса на колективното изпълняване на програмния продукт и др.**

3. Неструктурно програмиране (Unstructured Programming)

- Нарича се още **просто, последователно** или **императивно** програмиране.
- Исторически, това е първият подход за разработка на софтуер.
- Обикновено, когато човек започне да учи програмиране, той използва именно него, създавайки елементарни, неструктурирани програмки, съдържащи само една „main” функция (програма): последователност от оператори, които модифицират данните, които от своя страна са глобални.

3. Неструктурно програмиране (Unstructured Programming)



Главната програма директно оперира върху глобалните данни

Недостатъците:

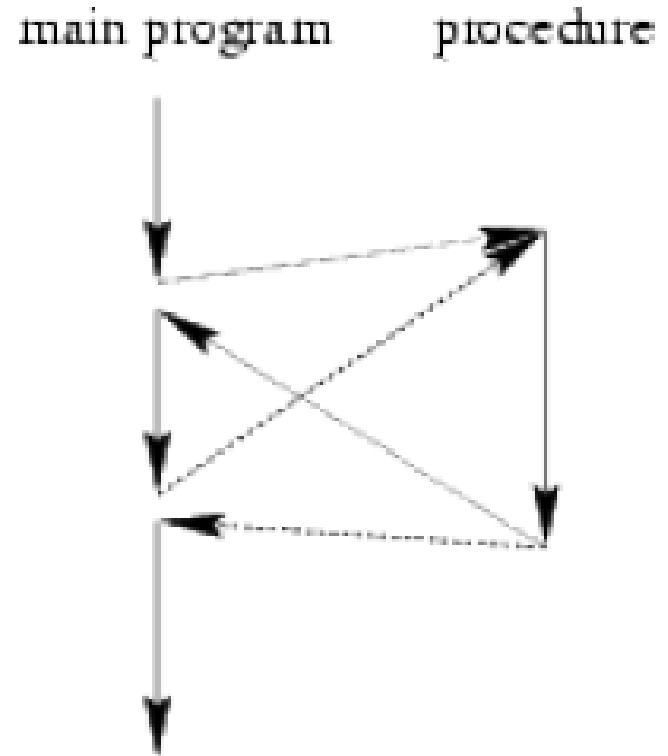
- Когато програмата стане дълга, получава се SpaghettiCode.
- Често в една такава програма, една и съща последователност от оператори се среща многократно.

Така, естествено се е стига до идеята за отделяне и именоване на тази последователност и за създаване на техника за многократно извикване и връщане, т.е до процедурния подход за тазработка на софтуер.

4. Процедурен подход за разработка на софтуер

- При процедурно програмиране, вие събирате една последователност от оператори в едно цяло, наречено **процедура, подпрограма, метод или функция**.
- Използва се „извикване” (call) на процедура за да се изпълни последователността от оператори.
- В съвременното програмиране процедурата може да има няколко изходни точки (return в C-подобните езици), няколко точки за вход (с помощта на yield в Python или статични локални променливи в C++), да имат аргументи, да връщат резултат от изпълнението си, да се самоизвикват (рекурсия) и т.н.

4. Процедурен подход за разработка на софтуер



- Процедурните програми представляват **съвкупност от процедури, които се извикват една друга, с предаване на данни между тях.**
- След изпълнението на процедурата управлението се подава на оператора след извикващия (след call оператора).

4. Процедурен подход за разработка на софтуер

Предимства, спрямо предния подход:

- Програмите стават по-структурирани и освободени от грешки.**
- Възможност да се използва повторно същия код (описан в процедурата) на различни места в програмата, без да го копирате.**

5. Модулен подход за разработка на софтуер

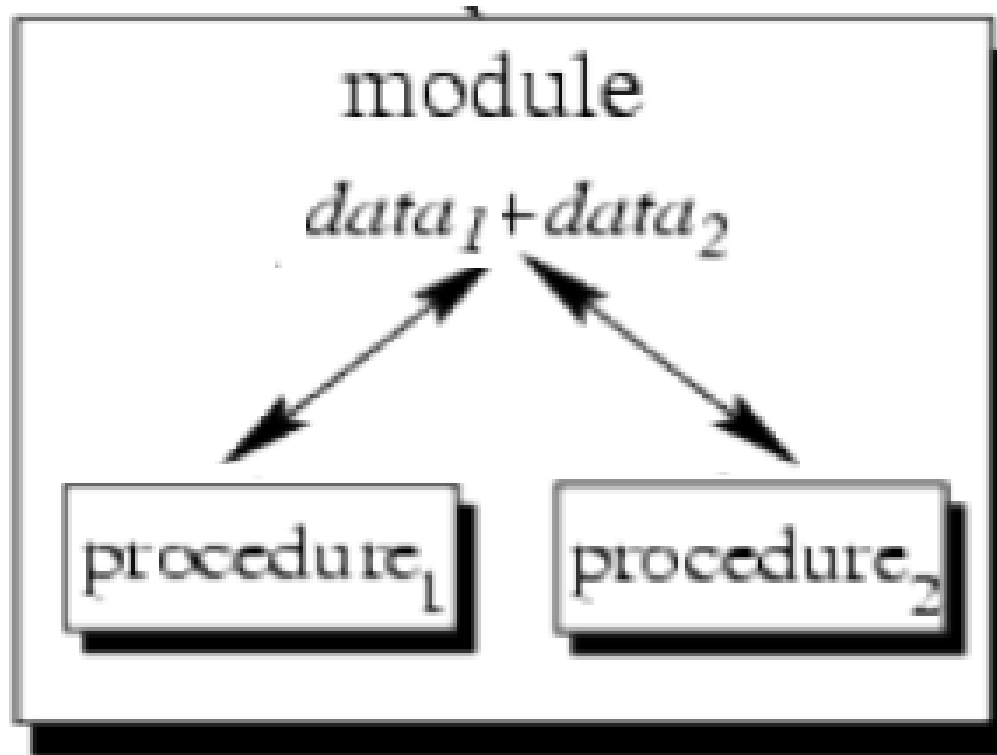
Причини за поява на МП

- Резултатът от процедурния подход е програма, която е разделена на малки парченца код, наречени процедури.
- За да се разреши обаче, **използването на дадена процедура или процедури и в други програми**, те (процедурите) трябва да са групирани заедно, в отделен файл. Така естествено се стига до идеята за модулно програмиране. Модулното програмиране е подход, който позволява групиране на процедури (и не само) в модули.

- Изказаното не е единствената причина за поява на модулния подход. **Нарастващата сложност на програмите** е основната причина разработчиците да търсят нови начини за разделяне на кода. Използването на процедурен подход се оказва недостатъчно от един момент нататък.
- Поглеждайки напред трябва да отбележим, че модулите също в един момент се оказват неспособни да отговорят на нарастващата сложност на програмите. Появяват се **пакети, класове, компоненти**.

Модул при МП и негови характеристики

- Модулът е **именована част** на ПС, която се **компилира самостоятелно** и се явява **строителен блок на физическата структура на ПС**. Какво има вътре в модула - една или множество процедури (функции), императивен код, един или множество класове – това не важно.



Модулът се характеризира:

- **функционална цялостност и завършеност** - всеки модул реализира цялостно и напълно една функция;
- **с един вход и един изход** – на входа се получава един набор от входни данни, изпълнява се обработка и се връща един набор от резултати (изходни данни), тоест реализира се принципа IPO – вход–процес–изход;
- **логическа независимост** —резултатът от работа на модула зависи само от **входните данни, но не зависи от работата на другите модули;**

- **слаба външна и висока вътрешна свързаност (**Low Coupling and High Cohesion**)**, т.е трябва да са налице слаби информационни връзки между модулите и силни информационни връзки вътре в модулите;
 - модулите трябва да са в максимална степен независими информационно, т.е обменът на информация между тях трябва да е минимизиран (**информационна затвореност на модулите – Low Coupling**), същевременно:
 - да е налице висока вътрешна свързаност (висока кохезия – **High Cohesion**). Идеалният модул: минимална външна и максимална вътрешна свързаност.
- **минимизиран размер на модула** - по възможност минимизиран (10 до 1000 оператора);

Примери за модулни решения (High Cohesion, Low Coupling):

- **Вътрешна свързаност «По съвпадение».** В своя първоначален вид, модулът представлява съвкупност от **функции** и **данни**, които (функции и данни) решават задачи от някакъв тип:
 - Например, в даден модул могат да са събрани функциите, обезпечаващи вход-изход.
 - В друг модул могат да са функциите, позволяващи да се изпълняват операции над низове.
 - Трети модул съдържа функции за изпълнение на операции над геометрични обекти и т.н.

Забележка: Счита се, че «по съвпадение» не е най-удачното решение за групиране на функциите в модул (**Cohesion=1**), поради високата вероятност от грешки при смяна на интерфейса за една от функциите.

- **«Временна» вътрешна свързаност (Cohesion=3).** Частите на модула не са функционално свързани, но са необходими в един и същи период от работата на системата. Например, в модула «Утро» могат да бъдат групирани функции: «миене», «обличане», «закуска».

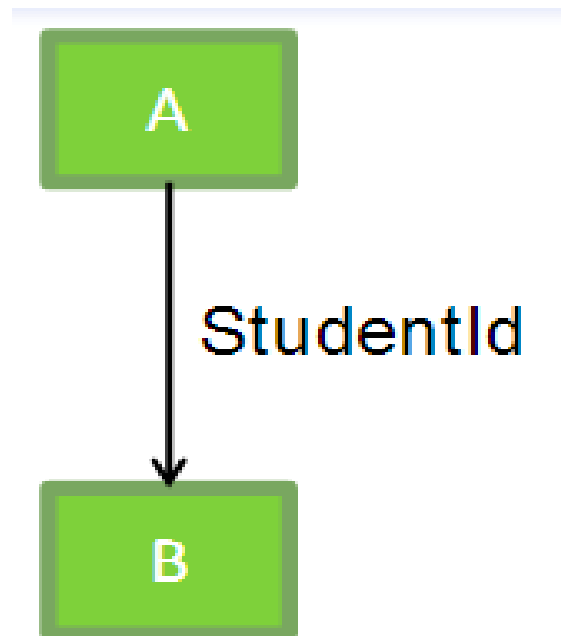
- **«Процедурна» вътрешна свързаност (Cohesion=5).** Частите на модула са свързани по реда на изпълняваните действия, реализирайки някакво цялостно поведение (признак за небрежно проектиране).
- **«Комуникативна» вътрешна свързаност (Cohesion=7).** Частите на модула са свързани по данни (работят с една и съща структура от данни).
- **«Последователна» вътрешна свързаност (Cohesion=9).** Изходните данни на една част се използват за входни на друга част на модула.
- **«Функционална» вътрешна свързаност (Cohesion=10).** Частите на модула заедно реализират една функция.

Функционална	Най-висока
Последователна	
Комуникативна	
Процедурна	
Временна	
Логическа	
По съвпадение	Най-ниска

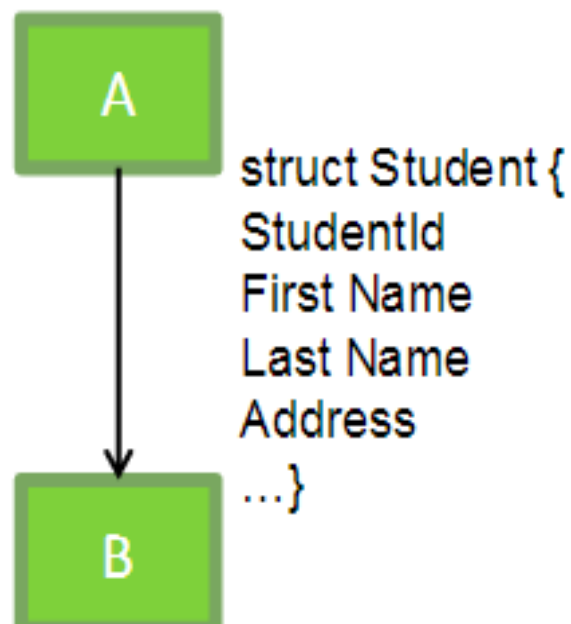
Свързаност между модулите

- **Външна свързаност «по данни».**

Комуникацията между модулите е изключително чрез обмен на прости данни, чийто обем трябва да се минимизира.



- **Външна свързаност «по образец».** В качеството на параметри за комуникация се използват **структури от данни**. Тъй като в структурата може да има излишни данни, то тази свързаност е по-малко предпочитана от предната (вж. Предния слайд).



- **Външна свързаност «по управление».** Модул А явно управлява функционирането на модул В, изпращайки му сигнали за управление (н.р. Установяване на стойност на някакъв флаг или превключвател).
- **«Обща» външна свързаност.** Модулите имат възможност да четат или модифицират някакъв външен ресурс (таблицы от БД, глобални данни, файлове)

- **Външна свързаност «по съдържание»**
имаме в този случай, когато един модул А има възможност директно да модифицира данните на друг модул Б или има пряка точка за вход от средата на метод в единия модул А в изпълнимия код на другия модул Б (посредством Goto и етикет в изходния код на модул А).

Тип външна свързаност

По данни

най-ниска

По образец

По управление

Обща

По съдържание

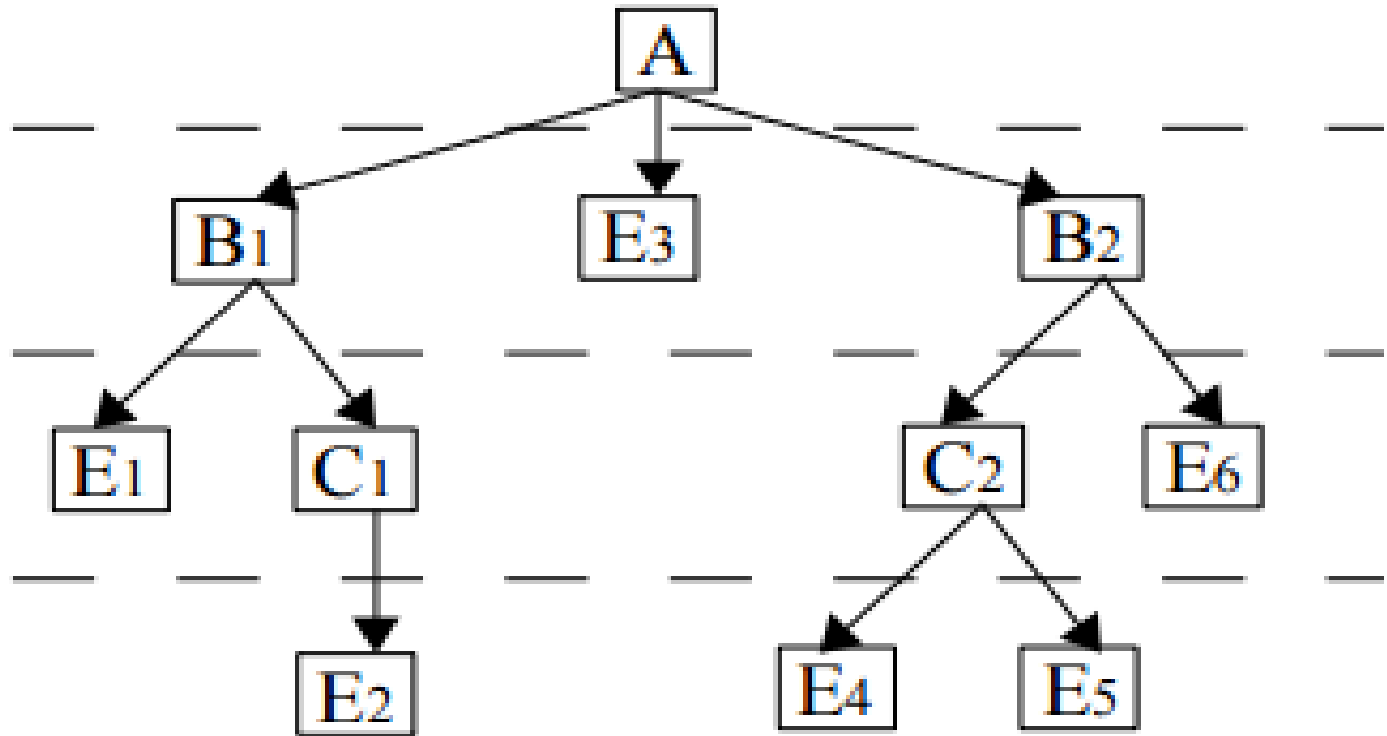
най-висока



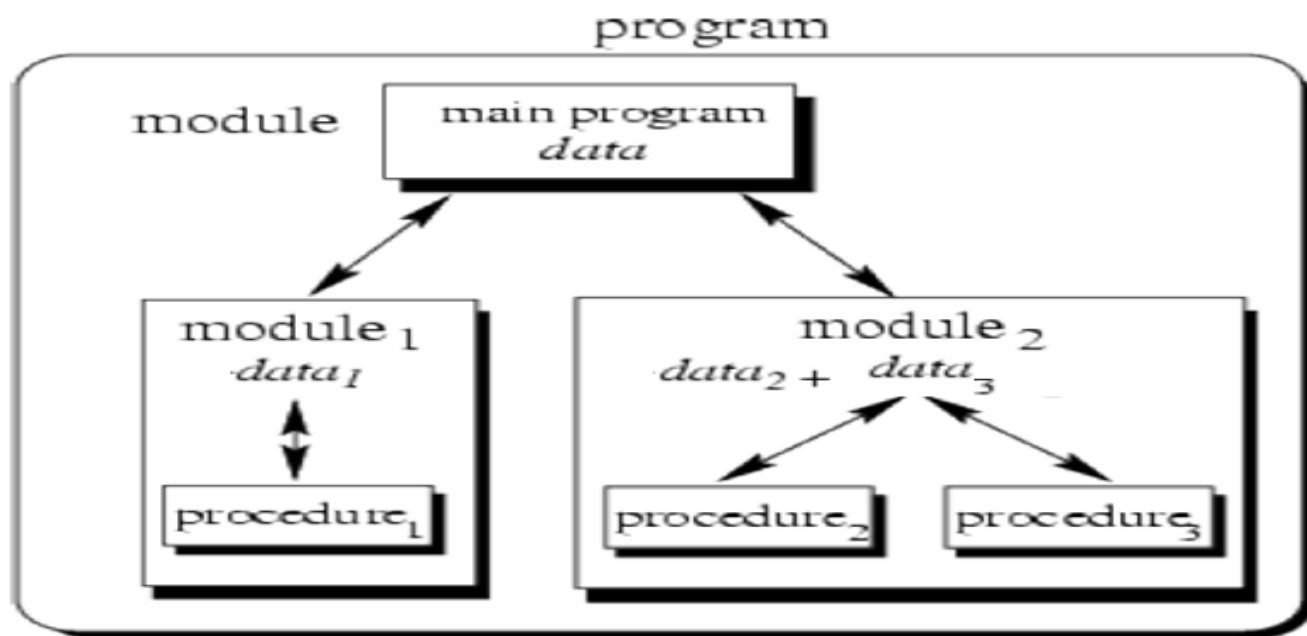
- Типични езици, имплементиращи МП са Modula 2, Delphi (Object Pascal) и др. В Object Pascal, например, параметрите за комуникация са описани в специална секция, наречена интерфейс («interface») на модула.

Декомпозиция при МП

- **Функционална декомпозиция:** Низходящо проектиране (Top-Down Design) за определяне състава и подчинеността на функциите, които програмата трябва да реализира:



- Функцията на най-високо (върхово) ниво се обезпечава от **главния модул**: той управлява изпълнението на нискостоящите функции, които се реализират чрез **подчинените модули**.



- Следва групиране на останалите функции, които програмата трябва да реализира в **подчинени модули** (избирайки тип свързаност).

Предимства на МП:

- повишаване на ефективността на производствения процес:
 - ранно разпаралелване - улеснява се груповата разработка;
 - всеки модул може да се проектира, програмира, компилира и тества отделно;
 - улеснява се развитието и подмяната на отделни модули на крайния ПП;
 - възможност за формализация и прилагане на математически методи за функционална декомпозиция;
- за първи път се появява сериозна:
 - **енкапсулация** – скриване на информацията от външния свят;
 - **пре-използване на код**;
- удобство за комбиниране с други технологии.

Недостатъци на МП:

- Крайният резултат силно зависи от използваните методи за функционална декомпозиция.
- Подходът изисква сериозно планиране и висока степен на дисциплина.
- Контролът на качеството се осъществява трудно.

Модули/класове

Различията между класове и модули:

- класовете могат да се инстанциират за създаване на обекти,
- класовете могат да наследят поведение и данни от друг клас,
- полиморфизмът позволява взаимоотношенията между инстанции на класа да се променят по време на изпълнение, докато отношенията между модулите са статични.

Приликите между класове и модули са:

- И класове и модули могат да бъдат използвани, за да се скрие информация за имплементацията от погледа на публиката.
- И класове и модули могат да образуват йерархия (от модули/класове).

6. Подходи Top-down и bottom-up

- Подходите **top-down** и **bottom-up** играят ключова роля в процеса на разработване на софтуер (software development process).
- При подход **отдолу-нагоре (bottom-up)** се тръгва с вече готови или адаптирани базови елементи (функции, модули и др.) на системата. Тези елементи се свързват заедно, за да образуват по-големи елементи (модули, подсистеми и др.), които след това на свой ред се свързват и т.н, докато се образува цялостна система.

Предимства на подход отдолу-нагоре :

- Възможност за използване на библиотеки от елементи,
- възможност за използване на опита от разработката на подобни системи,
- възможност за проектиране на ефективно управление на модулите,
- във всеки момент съществува някаква функционалност.

Недостатъци на подход отдолу-нагоре:

- Адаптирането на елементи понякога е по-трудно от създаването на нови, а групирането не винаги води до система, напълно съответстваща на техническото задание.

Подход Top-down

Top-down design

- Започва с най-обща спецификация на системата: специфицират се, но не подробно всички подсистеми на първо ниво.
- Всяка подсистема се детайлизира постепенно, на много нива, дотогава докато по натъшната детайлизация стане невъзможна - получават се базови елементи - функции, модули и др.

Top-down programming

- Започва се с написване на кода на основната процедура (`main()`), в която се посочват имената на всички основни функции, от които тя ще се нуждае.
- Следва преглеждане на изискванията за всяка от обявените функции в `main()` и процесът се повтаря, тоест написва се кода на обявените функции, като за всяка се обявяват имената на функциите, от които тя ще се нуждае.
- Процесът завършва, когато всички обявени функции са кодирани.

Предимства на top-down подход:

- Отделяне на работата на ниско ниво от тази по-високо ниво естествено води до модулна структура на програмата.
- Модулната програма позволява самостоятелна и паралелна разработка на отделните модули.
- Кодът лесно се проследява, тъй като е написан методично и целенасочено.

Недостатъци на top-down подход

- Нямаме функционалност до пълното разработване на всички функции от ниско ниво.
- Проектното решение е уникално, трудно се формализира, често не позволява използването на библиотека от готови модули, не се отчита опита от проектирането на подобни системи.

7. Структурен подход за разработване на софтуер (СПРС)

Структурният подход е комбинация от подходи:

- Функционална декомпозиция
- Низходящо проектиране (top-down design)
- Постъпкова детайлизация (stepwise refinement) на функциите
- Модулност на програмите
- **Структурно програмиране**

Извод: Структурният подход $\neq$ Структурно програмиране

Цели на **структурния подход**:

1. Да се дисциплинира и улесни процеса на създаването на сложни програми, чрез замяна на творческия подход с конструктивен
2. Да се «избавим от лошата структура на програмите» (SpaghettiCode)
3. Да се улесни разбирането, съпровождането и модифицирането на програмите

Началото

- Първите работи, свързани със структурния подход са свързани със статии на Дейкстра (1965) и Боем (1966).
- Дейкстра пръв обръща внимание на това, че опитните програмисти избягват използването на оператора GO TO в своите програми, а ги строят като съединение или влагане на 3-4 конструкции, всяка с един вход и един изход:
 - **линейна последователност от оператори,**
 - **условен преход,**
 - **цикъл и**
 - **обръщение към процедура.**
- Тази идея се развива: възприема се, че е намерен подход за създаване на програми, които са:
 - **разбираеми,**
 - **лесни за изучаване или променяне.**
- Резултатът от структурното програмиране (Structured Programming) е контраст на SpaghettiCode ("SpaghettiCode" - код, който няма проста логическа структура).

Проект на супер програмиста

- **Структурният подход** основно се свързва с далечната 1969г., когато по време на една научна конференция, спонсорирана от НАТО и свързана с техниката на програмирането, представители на IBM описват експеримент, известен като “проект на супер програмиста”. В този експеримент на един програмист, Dr. Harbar Mills, се възлага едно доста “невъзможно” задание (което би могло да се изпълни от 60 човека за шест месеца), което той изпълнява сам за шест месеца. Това значи увеличаване на производителността 60 пъти. Това той постига, като предлага и използва някои нови идеи при организацията на проекта и изпълнението му и най-вече **идеите за проектиране отгоре надолу и структурно програмиране.**

- Оттогава насам, независимо от критиките и появата на редица нови подходи, най-известният от които е обектно ориентирания, структурният подход остава да е един от най-често и широко използваните технологични подходи за разработване на софтуер.

!!!Структурният подход за разработване на софтуер (СПРС) не е само структурно програмиране!

Той включва още:

- **Функционална декомпозиция** на проблема
- **Низходящо, Постъпково проектиране** (**Stepwise Refinement и Top-Down design**) на функциите
- **Едновременно** проектиране на алгоритъма и данните;
- **Модулност на програмите**

Функционална/Обектно-ориентирана декомпозиция

Основната разлика между двата основни подхода за разработване на софтуер - структурен и обектно-ориентиран се заключава в **различните начини за декомпозиция на системата** [Вендров 2000].

- При **СПРС** имаме **функционална декомпозиция**: системата се разглежда в термините на **йерархия от функции с предаване на информация между тях**.
- При **ООПРС** имаме **ОО декомпозиция**: структурата на системата се представя в термините на **свързани помежду си обекти** (съответстващи на обектите в реалния свят и на връзките между тях), а поведението на системата - в термините на **съобщения между обектите**.

“Низходящо постъпково проектиране” и “Структурно програмиране” на функциите

Низходящо постъпково проектиране” и “структурното програмиране” вървят заедно:

1. **Низходящо постъпково проектиране**: имаме постъпково движение (**Stepwise Refinement**) от общата формулировка (описана словесно) на алгоритъма към словесната формулировка на съставляващите го действия (**Top-Down Design**), при това;
 - На всяка следваща стъпка на проектиране, една словесна формулировка на алгоритъма се замества със структурна конструкция (една от 3-те или 4-те възможни конструкции), а всички останали словесни формулировки остават в неформален вид, което предполага аналогични стъпки за тяхното проектиране (**Stepwise Refinement**);.

“Низходящо постъпково проектиране” и “Структурно програмиране”

Следва:

2. **структурно програмиране**: процес на **кодиране**, пак постъпково, в низходящ порядък, започвайки пак от най-външната структурна конструкция на алгоритъма (представена графично или като псевдокод) към най-вътрешната.
- Това е пак **низходящ постъпков** (на дадена стъпка, само една замяна) процес на замяна на всяка от структурните конструкции, представящи алгоритъма с една от 3 или 4 формални конструкции в езиците за програмиране, съответстващи на последователност, разклонение (if, case), цикъл (for, while, do) и извикване на процедура (call).

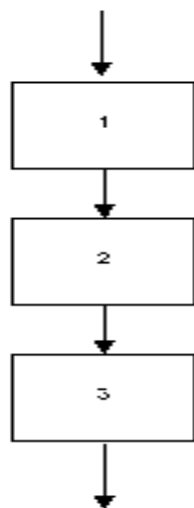
- При това, синтаксическата вложеност на програмните конструкции съответства на последователността на тяхното проектиране.
- Стига се до програма, в която принципиално отсъства оператор за преход `goto`, ето защо структурното програмиране се нарича понякога **програмиране без `goto`**.

Освен "GoTo" оператор, структурно програмиране забранява още:

- Използване на "break" или "continue" извън тялото на цикъл.
- Множество изходни точки във function/procedure/subroutine (тоест, множество "return" оператори).
- Множество входни точки във function/procedure/subroutine.

Структурни конструкции

Линейна последователност от оператори:



Псевдо код:

statement-1

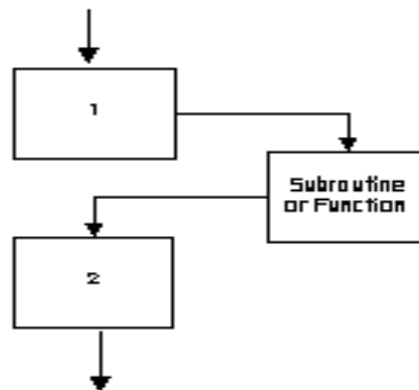
statement-2

statement-3

Извикване на процедура ('gosub', но не 'goto'):

Псевдо код:

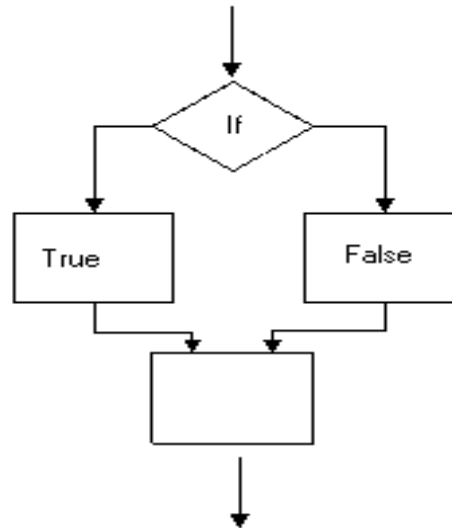
gosub



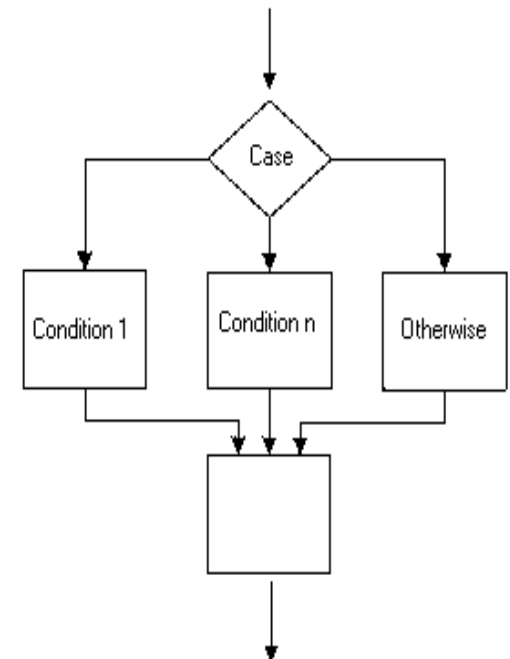
Конструкция за разклонение (т.е. "if", "case" оператори)

Псевдокод:

1. if condition
 true-statement
 else
 false-statement



2. case fieldname
 value1: statement-1
 value2: statement-2
 value3: statement-3
 otherwise: other-statement



Конструкция за цикъл: for, while or do..until

Псевдо код:

1. for (initial-value, final-value, increment)

statement-1

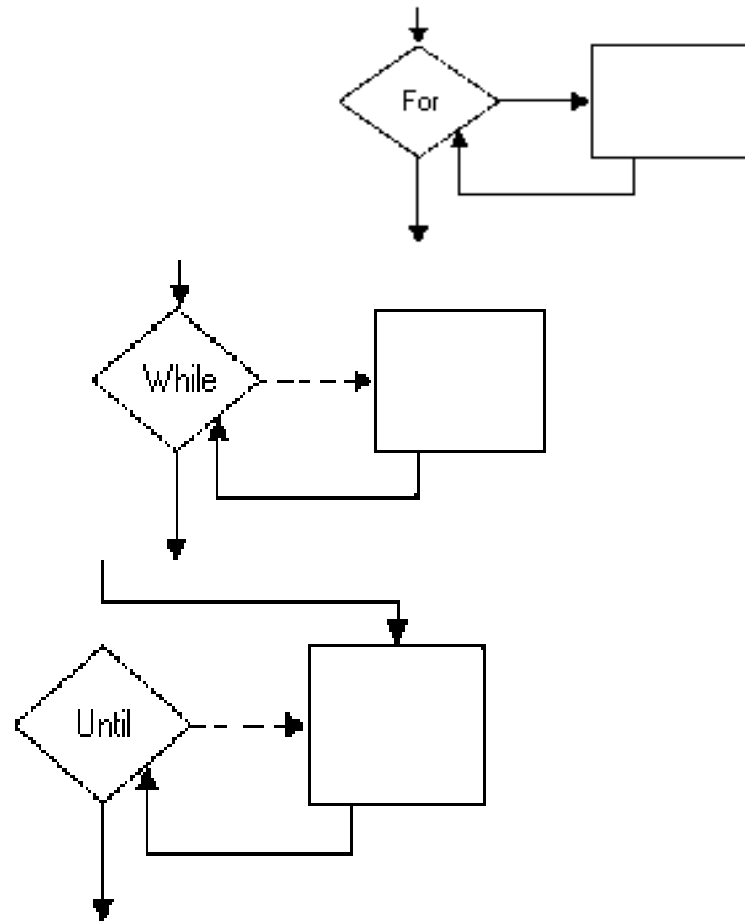
2. while not condition

statement-1

3. do

statement-1

until condition



- Вижда се, че всички конструкции имат един вход и един изход. При структурното програмиране, програмистът мисли като конструктор, на разположението на който има някои (неголямо число) напълно определени типови конструкции (структури).
- Разполага (или са зададени) с правилата за тяхното манипулиране и структурите могат да се съчленяват, да се влагат една в друга, да се разлагат на съставни.

- Милс, Ашкрофт и Мана посочват че всяка „правилна“ (определени са изискванията към такава програма) неструктурна програма може да бъде трансформирана в еквивалентна на нея структурна програма.

- Трансформирането е смислено:
 - ако се налага модифицирането на съществуваща програма с усложнена или объркана управляваща структура;
или
 - ако новоразработваните програми ще са структурни и след преобразуването ще се постигне унифицирано представяне (и документиране) на всички програми.

Едновременно проектиране на алгоритъма и структурата от данни.

- При низходящата постъпкова детайлизация на програмата естествено възниква необходимост от структури от данни и променливи при прехода от неформалните описания към конструкциите на езика, тоест процесите на детайлизация на алгоритъма и данните вървят почти паралелно.

За тестването при СПРС

- Може да се заключи, че тестване при тази технология може да стане едва, когато цялата програма е кодирана. Но това не е така. Тестване може да се извърши дори и ако разработката на програмата не е изцяло завършила, тоест дори ако на ниските нива има все още некодирани участъци.
- Тогава, на ниските нива могат да се поставят „заглушки“, произвеждащи един и същ очакван резултат, и така да се тества. Този подход за тестване се прилага и при модулния подход за разработване на софтуер.
- От казаното може да се заключи също и че **тестването може да се извършва в някаква степен паралелно с нейната детайлизация.**

Основни преимущества на структурното програмиране:

1. Тъй като програмите се **“конструират”** от типови елементи, това води до:
 - **намаляване на изискванията към квалификацията на проектантите и програмистите (т.е не е висока).**
 - **намаляване на усилията за тяхното проектиране, кодиране, тестване и съпровождане, а оттам:**
 - **повишаване на производителността на проектантите и програмистите**
 - **намаляване на цената на разработката**
2. **Разработка на качествена документация:**
 - **документирането следва структурирането на програмата и има възможност за автоматизиране.**

Недостатъци:

- Основен недостатък на структурното програмиране е необходимостта **от усвояване на тази специална програмистка техника и прилагането** и при създаване на всички програми. Консервативно настроените програмисти се съпротивляват на всяка промяна в стила им на работа и обикновено разглеждат опитите за, въвеждане на стандарти като посегателство на личната им творческа свобода.)
- Друг недостатък е, че в някои случаи **структурните програми изискват по-големи изчислителни ресурси** (памет и време) от съответните неструктурни програми.
- “конструираните” програми имат по-лоши експлоатационни характеристики, програмите са по-дълги, вероятността за допускане на грешка е по-голяма, като следствие намалява надеждността,
- не могат да се разработват “уникални” решения и др.

8. Обектно-ориентиран подход за разработване на софтуер

- Теорията и практиката на обектно ориентирана (ОО) парадигма са изключително развити днес, те са обект на задълбочено разглеждане в друга ваша дисциплина.
- Целта на тази лекция е да се спрем на **основните характеристики на ОО-подход (ООП) за разработване на софтуер, с оглед на особеностите, които го разграничават от други подходи.**

Процедурни/Обектно-ориентирани програми

Обектно-ориентираното програмиране е технологиите, които разгледахте досега, в това число и на **процедурното програмиране**.

- Процедурните програми представляват съвкупност от процедури, които се извикват една друга. При процедурните програми, данните и процедурите за тяхната обработка формално не са свързани, за разлика от ОО програми.
- Проблемът при процедурното програмиране е, че **преизползваемостта на кода е ограничена** – само процедурите могат да се преизползват, а те трудно могат да бъдат направени общи и гъвкави.

Структурен/ОО подход

- При **структурния подход** софтуерната система се разглежда като съвкупност от компоненти, всяка от които реализира определена функция (функционална декомпозиция). Състоянието на системата е централизирано и се разделя между функции, опериращи върху това състояние.
- При **обектноориентираното проектиране** системата се разглежда като съвкупност от обекти, които комуникират помежду си чрез съобщения. Състоянието на системата е децентрализирано и всеки обект управлява собственото си състояние. Обектите имат множество от атрибути, определящи състоянието им, и операции върху тези атрибути.

Структурен/ОО подход

- ОО-подход се явява развитие на структурния подход. Идеята е да създаде предпоставки за:
 - по-лесно разработване, поддържане и еволюция на големи по мащаб ПС.
 - създаване на код, който е по-лесен за изучаване от начинаещи програмисти (сравнен със структурния)
 - създаване на по-лесен за разбиране код

ООП - Основни понятия

- Под **обект** се разбира **познаваем предмет, елемент или същност** (реална или абстрактна), с важно функционално предназначение в разглежданата приложна област.

Всеки обект има **състояние, поведение и индивидуалност**:

- **състояние** - всички **възможни атрибути на обекта и текущите стойности на тези атрибути**. Атрибутите са отличителните черти на обекта, например обект студент (Student1) има атрибути факултетен номер, име, възраст (fnum, name, age) и др.

ООП - Основни понятия

- **поведение** - това са операции или услуги, които променят един или няколко атрибута на обекта.

и

- **индивидуалност** - това са тези атрибути на обекта и техните стойности, които го отличават от други сродни обекти.
 - Например, такъв атрибут е факултетния номер за обект студент: обектът **Student1** има стойност „**123456**” за атрибута факултетен номер (fnom), обектът **Student2** – “**234567**”, но и **Student1** и **Student2** могат да имат еднакви стойности (н.р 22) за атрибута age, както и еднаква стойност за атрибута name (н.р Иван Иванов).

ООП - Основни понятия

- Състоянието и поведението на сходни обекти определят общ за тях **клас**. Понятието **клас** е основно за ОО-подход.
- Класът, например **Student**, може да се разглежда като обобщено описание на съвкупност от подобни обекти: **Student1**, **Student2** и др.
- **Обектите (objects)** са екземпляри (инстанции) на класовете. Например обект «**Student1**» е екземпляр (инстанция) на клас **Student**, обект «**Student2**» също е екземпляр (инстанция) на клас **Student**.

Обектно-ориентирано програмиране (ООП)

- Операциите и атрибутите на класа могат да бъдат:
 - видими само в областта на класа, в който са декларирани и в наследниците му (private/protected),
 - или видими за всички останали класове (public).
- **Обектите** на класа **наследяват** неговите атрибути и операции.

```
class Student
```

```
{ private string name; //поле  
  private int age;      //поле  
  public Student(string n, int a)
```

```
//конструктор
```

```
{ name = n; age = a; }  
  public string Name  
  {
```

```
// свойство Name с достъп само за четене
```

```
    get { return name; }  
  }
```

```
  public int Age  
  {
```

```
//свойство Age - достъп само за четене
```

```
    get { return age; }  
  }
```

```
  public void DisplayData()
```

```
//метод
```

```
{Console.WriteLine(name + " " + age);}  
}
```

Примерно описание на клас
Student в UML нотация и
език за програмиране (C#)

Student

Class

Fields



age



name

Properties



Age



Name

Methods



DisplayData



Student

Йерархия на класовете



- Може да се създаде **йерархия на класове**, като **подкласовете** наследяват атрибутите и операциите на **супер-класа**, но могат да имат и специфични за тях атрибути и операции.
- **Суперклас** (напр. Транспортно средство) е съвкупност от класове, а **подклас** (напр. Автомобил, Автобус, Трактор) е екземпляр на супер-класа.

Йерархия на класовете



- Допуска се и „**множествено наследяване**“ от няколко различни суперкласа. То е полезно заради по-големия брой наследявани свойства, но затруднява проследяването на връзките в йерархията.

Основни принципи на ООП

Капсулация (Encapsulation) в класа на данните (атрибутите) и операциите върху тях и същевременно и предоставяне на прост и ясен интерфейс за работа с тях. Това има следните преимущества:

- подробностите на вътрешната реализация остават скрити от външния свят;
- данните и операциите са обединени в едно именовано цяло — класа, което улеснява повторното му използване;

!!!

Обектно-ориентираното
програмиране по своята природа се
явява **модулно** – «енкапсулацията»
е основно доказателство за това.

Основни принципи на ООП

Наследяване (Inheritance)

Същността на наследяването е, че всеки подклас придобива автоматично (т.е наследява) всички атрибути и операции на съответния супер-клас.

- Така се осигурява **повторно използване** на проектираните и реализирани вече структури данни и алгоритми.
- **Улеснено е внасянето на изменения**, защото те се правят само в съответното ниво, в йерархията от класове и чрез наследяването се разпространяват.

Основни принципи на ООП

Полиморфизъм (Polymorphism)

Смисълът на полиморфизма може да бъде изразен накратко със следната фраза - **"Един интерфейс, множество от различни реализации"**. Чрез полиморфизма се постига по-голяма абстракция и по-лесно повторно използване на кода.

Примери в ОО езици: различни операции на класовете да имат едно и също име; **специфична имплементация на някакво абстрактно поведение в класовете**, наследяващи даден клас. Така се осигурява настроена емост и гъвкавост, защото с унифицирано извикване (тоест един и същ интерфейс) могат да се реализират специфични обработки (т.е множество различни реализации).

Основни принципи на ООП

Абстракция (Abstraction)

- да виждаме един обект само от гледната точка, която ни интересува, и да игнорираме всички останали детайли.

Обектно-ориентирани езици за програмиране

- И така, за да бъде един програмен език обектно-ориентиран, той трябва не само да позволява работа с класове и обекти, но трябва да дава възможност за имплементирането и използването на коментираните по горе принципи на ООП:
 - Капсулация
 - Наследяване,
 - Полиморфизъм
 - Абстракция.

Съобщения

Взаимодействието между обекти при ОО подход става чрез **съобщения**.

- Какво е «съобщение»?
 - **специален обект**, т.е «**обект-съобщение**».
 - **външен метод**, чрез извикването на който се реализира взаимодействието между обектите.

- **специален обект, т.е «обект-съобщение».** В някои езици (Smalltalk, Ruby, Python) взаимодействието между обектите става чрез пренос на специален обект - «обект-съобщение» - това е отделна, абсолютно автономна изчислителна единица.
- **външен метод.** В повечето ОО езици за програмиране (C++, Object Pascal, Java, C#, Oberon-2) се използва именно тази концепция - «изпращането на съобщение се реализира чрез извикване на външен метод» (т.е обектите имат на разположение външни методи, чрез извикването на които се реализира взаимодействието между тях).

Обектно-събитийната парадигма (ОСП)

- **Обектно-събитийната парадигма** е едно от развитията на обектно-ориентираната парадигма.
- Новото понятие тук е «**събитие**».

Събитие в ООП

- **Събитие в ОСП:** Събитието е <абстракция на **инцидент** или на **сигнал** в реалния свят. Например събитие **button1_Click()** е абстракция на “**реално щракане върху нещо**” (например върху “**бутон**”).
- При **събитие** върху “**нещо**”-то се генерира **съобщение**, което се прихваща и обработва от т.н «**обработчик на събитието**» - това е метод в програмата, разработен специално за обработка на даденото събитие.

Събитие в ООП

- Най-нагледно събитията са представени в потребителския интерфейс, когато всяко действие на потребителя може да породии верига от събития върху обект, които приложението трябва да обработи.

Взаимодействието на обектите

- Взаимодействието на обектите се осъществява чрез **съобщения** (messages), които се генерират в резултат на **събития** върху “обектите-податели” на съобщението.
- **Съобщението** (от страна на **обекта - подател**) предизвиква реакция и промяна в състоянието на “**обекта – получател**” (“обекта-получател” и “обекта-подател” могат да са един и същ обект).
- “**Обектът – получател**” изпълнява посочената в съобщението операция и връща управлението на обекта “**обекта-подател**”.

Пример: когато потребителят натисне `button1` надписът му се променя на “Click me”, в `label1` се извежда текст “Ivan”, а в `label2` – текст “Dragan”:

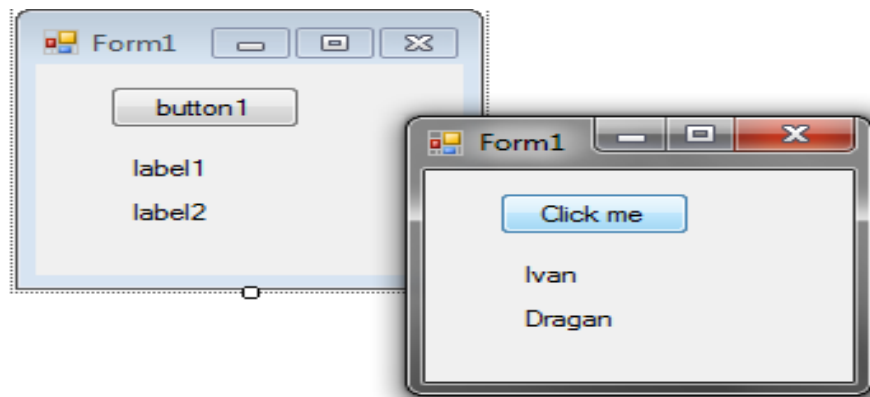
```
private void button1_Click(object sender, EventArgs e)
```

```
{ label1.Text = "Ivan";
```

```
label2.Text = "Dragan";
```

```
button1.Text = "Click me";
```

```
}
```



Когато потребителят натисне бутон (събитие `Click`) върху обект **`button1`** (подател на съобщението) се изпращат съобщения към обекти **`label1`**, **`label2`** и **`button1`** (получатели на съобщението). Резултатът от съобщенията е промяна на състоянието на обектите – получатели. Управлението винаги (след всяко съобщение) се връща в обекта подател (в случая - `button1`).

Въпроси

- Какви са изискванията към съвременните подходи за разработване на софтуер
- Каква е разликата между неструктурно програмиране (Unstructured Programming) и процедурен подход за разработка на софтуер?
- Какви са предимствата и недостатъците на модулното програмиране (Modular programming) ?
Има ли разлика между модул и клас?
- Какви са особеностите на Top-down и bottom-up design?

Въпроси

- Какво означава структурен подход за разработване на софтуер ? Кои са основните проблеми при определяне на изискванията? Каква е разликата между потребителски и системни изисквания? Има ли стандарти за документиране на изискванията? Кои са компонентите на аналитичния модел? Какъв подход за проектиране е типичен за структурния подход за разработване на софтуер и какви методи за описание на проекти познавате?

Въпроси

- Какво се разбира под структурно програмиране? Какви структури и операции се свързват със структурното програмиране? Кои са предимствата и недостатъците на структурното програмиране? Какви видове тестове се използват при този подход?

Използвана литература:

- Нели Манева, Аврам Ескенази, Софтуерни технологии, Анубис София 2001
- Нели Манева, Аврам Ескенази, Софтуерни технологии, КЛМН, София 2006
- Sommerville: Software Engineering , 9. ed. Addison-Wesley, 2011,
[http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20\(intro%20txt\)%20-%20I.%20Sommerville%20\(Pearson,%202011\)%20BBS.pdf](http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20(intro%20txt)%20-%20I.%20Sommerville%20(Pearson,%202011)%20BBS.pdf)
- <http://ermak.cs.nstu.ru/cprog/html/033.htm>